

What is MCP?

MCP (Model Context Protocol) is an open protocol that provides a **standard way for AI applications to connect with external tools, data sources, and services.**

It acts as a **bridge between an AI model and external systems** like databases, APIs, files, and applications.

Instead of every AI application building custom integrations for every tool, MCP provides a common communication standard.

MCP architecture consists of an AI application containing an MCP client, which communicates using the Model Context Protocol with one or more MCP servers that expose tools, resources, and prompts connected to external systems.

MCP Client	MCP Server
Lives inside AI application	Runs separately
Sends requests	Receives requests
Discovers available capabilities	Exposes capabilities
Executes nothing itself	Executes tools and fetches data
Can connect to many servers	Can serve many clients

? **MCP (Model Context Protocol):** A standard protocol that enables AI applications to communicate with external tools, resources, and prompts.

? **MCP Client:** Lives inside the AI application and communicates with MCP servers.

? **MCP Server:** Exposes tools, resources, and prompts to AI applications.

? **Tool:** Performs an action (e.g., `send_email()`).

? **Resource:** Provides readable data (e.g., `employee_handbook.pdf`).

? **Prompt:** Reusable instruction template.

? **MCP vs API:** MCP standardizes AI interactions with external systems and often uses existing APIs underneath.

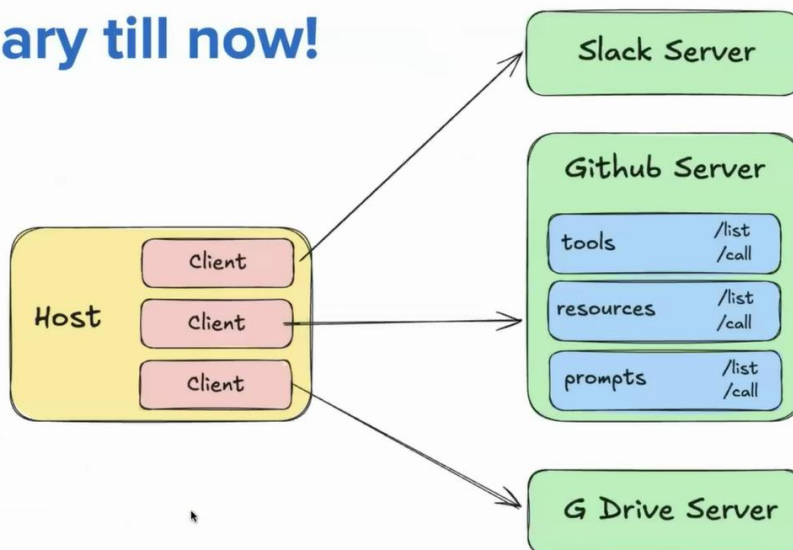
? **MCP vs Function Calling:** Function calling uses manually defined functions; MCP enables standardized discovery and use of capabilities across systems.

? **LangGraph + MCP:** LangGraph handles reasoning and workflow, while MCP handles communication with external services. Together they form a scalable architecture for AI agents.

Primitives - Standard Operations

- **Tools**
 - `tools/list` → Client asks the Server: "What tools do you provide?"
 - `tools/call` → Client tells the Server: "Please run this tool with these arguments."
- **Resources**
 - `resources/list` → Client asks: "What resources are available?"
 - `resources/read` → Client says: "Give me the content of this resource."
 - `resources/subscribe` / `unsubscribe` → Client subscribes or unsubscribes from updates
- **Prompts**
 - `prompts/list` → Client asks: "What prompt templates do you provide?"
 - `prompts/get` → Client fetches a specific prompt template.

Summary till now!



JSON-RPC 2.0

Definition

JSON-RPC 2.0 is a lightweight **remote procedure call (RPC) protocol** that uses **JSON** to exchange messages between a client and a server.

It allows a client to **call functions (procedures) on a remote server** as if they were local functions.

Why is it called JSON-RPC?

- **JSON** → Data is exchanged in JSON format.
- **RPC (Remote Procedure Call)** → Calling a function that executes on another machine.

Example in MCP

Imagine the AI wants to call a GitHub tool.

The MCP client sends something like:

```
</> JSON

{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "create_issue",
    "arguments": {
      "title": "Bug in login page"
    }
  }
}
```

The MCP server executes the tool and replies:

```
</> JSON

{
  "jsonrpc": "2.0",
  "id": 42,
  "result": {
    "status": "Issue created"
  }
}
```

? Bidirectional Communication

- REST is inherently **request-response**: the client asks, the server replies.
- MCP needs **two-way communication**: servers can send events, notifications, or streaming updates back to clients without waiting for a request. JSON-RPC supports this via **notifications** and **subscriptions**, while REST does not natively.

? Lightweight & Consistent

- REST requires defining endpoints (/users, /messages, etc.), HTTP verbs (GET, POST, PUT, DELETE), and often complex status codes.
- JSON-RPC uses a **single endpoint** with structured messages (method, params, id). This makes it simpler, more uniform, and easier to extend.

? Streaming & Real-time

- MCP often deals with **LLM pipelines, embeddings, and retrieval**, where streaming responses are critical.
- JSON-RPC can handle **progressive results** and **event-driven updates** cleanly. REST would need hacks like long polling or WebSockets bolted on.

❓ Transport Flexibility

- REST is tightly coupled to HTTP.
- JSON-RPC can run over **WebSockets, stdio, or other transports**, which is perfect for MCP since hosts and servers may run locally, inside containers, or across networks.

1. Local MCP Servers

Run on your own machine.

They communicate with the host using **stdio** (standard input/output).

Examples:

- File System Server
- SQLite Server
- Local Git Server

2. Remote MCP Servers

Run on another machine or in the cloud.

Communication happens over **HTTP, WebSockets**, or other network transports.

Examples:

- GitHub MCP Server
- Slack MCP Server
- Google Drive MCP Server

MCP lifecycle is a sequence of steps that define how Host, MCP client, MCP server establishes connection, exchange the context and tool request and terminate the connection during a session.

Initialization (Handshake)

- Host starts the MCP client. The client connects to the MCP server. Communication channel will be established between client and server.
- The client and server exchange protocol versions and capabilities.
- Version Negotiation: The client sends the **MCP protocol version** it supports. The server responds with the protocol version it has selected (typically the highest mutually supported version). If no compatible protocol version exists, initialization fails and the connection is terminated.
- Capability Negotiation: The client advertises the capabilities it supports. The server advertises the capabilities it supports (such as **tools, resources, prompts, logging**, etc.). Both sides determine which protocol features can be used during the session.

Capability Discovery

- The client requests available tools, resources, and prompts.
- The server responds with metadata describing them.

Operation Phase

- The LLM decides when it needs external information or an action.
- The host asks the client to invoke a tool or fetch a resource.
- The client sends an RPC request to the server.
- The server executes the request and returns the result.
- This request–response cycle can happen many times during the session.

Connection Termination

- The session ends when the host closes the client or the connection is terminated.
- Resources are cleaned up and the communication channel is closed.

Ping is a lightweight request/response method defined in MCP. It is used to check whether the other side of the connection (client or server) is still alive and the connection is response.

Useful for checking if the other side is up before full initialization.

If there's no activity for a while, a client may send periodic pings. Prevents the connection from being dropped silently by the OS, proxies, or firewalls.

Error Handling is how the Host(client) and Server signal that something went wrong with a request.

Unsupported or mismatched protocol version, calling a method for a capability that wasn't negotiated.

Invalid arguments to a tool, Internal server failure while processing a request, Timeout exceeded.

There are **two common ways**:

1. Configuration-Based Connection

The MCP server is defined in a **configuration file**, and the host reads this configuration when it starts.

For example:

```
</> JSON

{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/prudhvi/Documents"]
    }
  }
}
```

Or for a remote server:

```
</> JSON

{
  "mcpServers": {
    "github": {
      "url": "https://example.com/mcp",
      "headers": {
        "Authorization": "Bearer <token>"
      }
    }
  }
}
```

2. Connector-Based Connection

Instead of manually editing a configuration file, the host uses a **connector** to install and configure the MCP server automatically. It is easy, safe and consistent.

How it works

1. User selects an MCP server from a marketplace or integration page.
2. The connector handles installation, authentication (OAuth/API keys), and configuration.
3. The host automatically adds the server.
4. The host establishes the MCP connection.